

## Homework-7: GenAI & TensorFlow & Jax

**Deadline: November 20<sup>th</sup>, 11:59 PM ET**

Your homework submission should be on GitHub. Use the following GitHub classroom to access the assignment and create your assignment repository:

<https://classroom.github.com/a/gSzsJKyk>

- **Don't create a public repository in your own GitHub account. Use the URL above to create the GitHub repository for this assignment. It includes a starter ReadMe file.**
- On Gradescope, only submit a URL to your GitHub Classroom repository.
- **All screenshots must include your full name in a text editor.**
  - **If your output is not printed in a Jupyter notebook, take a screenshot of the output and make sure your name is written in a Text Editor in the screenshot (Don't photoshop your name).**
- Not following the submission guidelines may result in grading penalties.

Cite any external sources you use. External sources shouldn't exceed more than 30% of the final solution.

Submit the HW in one or more Jupyter notebooks (or a series of Python files with clear comments). If you need to include scripts outside Jupyter for a given question, note it in your notebook. If you need to include output or screenshots, you may include them in your notebook or attach them as separate image files using the naming convention qx.png (where x is the question number). If you have more than one image for a given question, name the files as qx\_1.png, qx\_2.png, etc.

**Q1 (30%)** In this question, you are asked to curate a text dataset of your own choice (at least 1MB, must be different from the Shakespeare dataset in lecture) and run the minimalistic GPT implementation in Lecture 17 on your dataset. You are asked to try 3 sets of hyperparameters (block\_size, n\_layers, n\_embd, n\_head) *with increasing values*. In your submission, **provide the 3 sets of hyperparameters and the corresponding text generations**.

*Note: The purpose of the question is to observe the Scaling Law that OpenAI discovered for language models,<sup>1</sup> that is, the larger the model (and dataset size), the better the generation quality.*

---

<sup>1</sup> See <https://arxiv.org/pdf/2001.08361> for details.

Please finish Q2 and Q3 of this homework with the provided starter code at [https://www.guannanqu.com/files/cmutoolchain/HW7starter\\_code.zip](https://www.guannanqu.com/files/cmutoolchain/HW7starter_code.zip)

**Q2 (40%)** In this question, you are asked to use TensorFlow keras to train a model for a computer vision task, MNIST-Fashion. The starter code for this question provided the training dataset (x\_train, y\_train), validation dataset (x\_val, y\_val), and test dataset (x\_test, y\_test).

- Q2-1 (20%) Create a model using tf.keras for this problem. You are free to choose your own model, but we provide a reference CNN model architecture that you may use. It consists of the following layers:
  - Conv2D with 32 filters (3×3), ReLU activation, input shape 28×28×1
  - MaxPooling2D with pool size 2×2
  - Conv2D with 64 filters (3×3), ReLU activation
  - MaxPooling2D with pool size 2×2
  - Flatten layer
  - Dropout with rate 0.1
  - Dense layer with 64 units, ReLU activation
  - Dense output layer with 10 units, Softmax activation

**Print out the model that you created (using the summary() function).** Use tf.keras to train your model on the training dataset with per-epoch validation on the validation dataset. **Include the printout** of the training process in your submission. Use tf.keras to calculate the test accuracy of your trained model on the test dataset. **Print out the test accuracy.**

*Hint: To get full credit, the dimension and activation of the last layer of your neural network must be compatible with your loss function.*

- Q2-2 (20%) Use Tensorboard to display the training progress in Q2-1. **Include screenshots of the loss and the accuracy for both training and validation.**

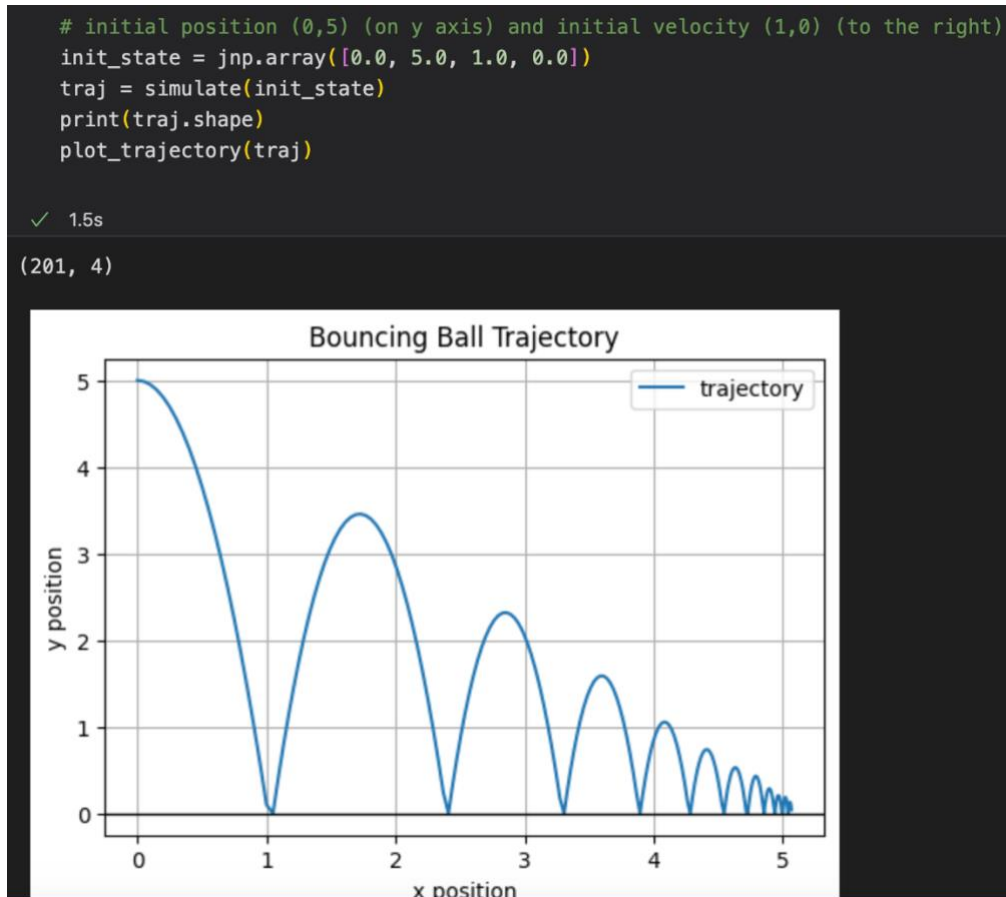
**Q3 (30%)** In this question, you will use JAX to perform gradient-based optimization through a physics simulator. The goal is to determine the initial velocity that causes a bouncing ball (modeled as a point with coordinates x,y) to land on a desired target. Specifically, the state of the ball is determined by a 4-dim vector (x, y, v\_x, v\_y), where (x, y) is the position of the ball; and (v\_x, v\_y) is the velocity of the ball.

You are provided with the following helper functions in the starter code:

- simulate(init\_state) — simulates the ball trajectory with initial state given by init\_state (i.e. a 4-dim vector specifying the initial x,y position and v\_x,v\_y velocity). It will simulate the ball's trajectory for 201 time steps and return a trajectory of dimension (201,4), with each row indicating the state of the ball at that time step. In the simulation, the ball follows the law

of physics (gravity), and whenever it hits the ground (the line of  $y=0$ ), the vertical velocity  $v_y$  is reversed, and both  $v_x$ ,  $v_y$  will shrink by a factor 0.8.

- `plot_trajectory(traj)` – this will plot the trajectory generated by the `simulate` function.
- An example usage of the helper functions is as follows.



You are asked to write a function that takes in two arguments:

- `pos_init`: 2-dim vector indicating the initial position of the ball
- `pos_target`: 2-dim vector indicating the target position of the ball

and returns an initial velocity, denoted as `optimized_v_init`, of the ball such that when starting with initial state as `(pos_init, optimized_v_init)`, the ball will land at the target position at the final step of the simulation. Intuitively, this means finding the correct direction and strength to throw the ball such that it will land at the target point.

In achieving the above, you are expected to define a loss function that measures how close the final point in the trajectory is to the target point. Then, conduct gradient descent of the loss w.r.t. the initial velocity such that the initial velocity is updated to minimize the loss.

In order to utilize the full power of Jax, you are required to **apply `jax.jit` to an appropriate function**. Failing to do so will result in a penalty.

In your submission, set `pos_init` and `pos_target` as

```
pos_init = jnp.array([0.0, 1.0])
pos_target = jnp.array([8.0, 0.0])
and print out the optimized_v_init returned by your function. Further, run the following code
new_init_state = jnp.concat([pos_init, optimized_v_init])
traj = simulate(new_init_state)
plot_trajectory(traj)
```

to simulate and **display the ball's trajectory** with the optimized\_v\_init returned by your function.

*Note: This homework is designed to give you hands-on experience with differentiable simulators, an emerging and rapidly growing area<sup>2</sup> at the intersection of machine learning, physics, and traditional engineering fields (mechanical engineering, biomedical, etc). Being able to compute gradients through simulations enables powerful applications in robotics, optimal control, and scientific discovery.*

---

<sup>2</sup> See e.g. <https://github.com/google/brax>